

Section 7 — Tic-Tac-Toe with Facial Controller

Player X = random bot, Player O = webcam facial controller (neutral→0, happy→1, surprise→2). Model used: **results/best_basic_model.keras** (test accuracy **0.7775**).

Trace of moves (from local python3 run.py)

Start:

```
| | | |
| | | |
| | | |
```

1. X (random) → (1,0)

```
| | | |
|X| | |
| | | |
```

2. O webcam: neutral + neutral → (0,0)

```
|O| | |
|X| | |
| | | |
```

3. X (random) → (2,2)

```
|O| | |
|X| | |
| | |X|
```

4. O webcam misread occupied cells; used text override → (1,2)

```
|O| | |
|X| |O|
| | |X|
```

5. X (random) → (0,1)

```
|O|X| |
|X| |O|
| | |X|
```

6. O webcam + text override → (1,1)

```
|O|X| |
|X|O|O|
| | |X|
```

7. X (random) → (0,2)

```
|O|X|X|
|X|O|O|
| | |X|
```

8. O text override → (2,0)

```
|O|X|X|
|X|O|O|
|O| |X|
```

Player O wins on the main diagonal (0,0)-(1,1)-(2,2).

How well did the interface work?

The webcam pipeline works: it crops a centered square, converts to grayscale, resizes to 150×150, expands to RGB, and classifies with the saved model. When a predicted cell was already taken, the built-in text override (type text, then 0/1/2) recovered cleanly and let the

game finish.

Webcam accuracy vs test-set accuracy?

Held-out test accuracy for the final model is **0.7775**. Webcam accuracy was lower in practice: lighting, framing, and expression intensity differ from the training images, so neutral was over-predicted until text override was used for some moves.

If not, why not?

Domain shift: training faces are tightly cropped and centered; live webcam frames include different illumination, head pose, and expression strength. Occupied-board retries also made wrong repeats more noticeable until overridden.

_get_emotion implementation

```
def _get_emotion(self, img) -> int:
    img = cv2.resize(img, image_size)
    img = np.stack((img, img, img), axis=-1)
    img = np.expand_dims(img, axis=0)
    prediction = self.model.predict(img, verbose=0)
    pred = int(np.argmax(prediction[0]))
    # Keras folder order is alphabetical (happy,neutral,surprise)
    # Assignment order is neutral=0, happy=1, surprise=2
    mapping = {0: 1, 1: 0, 2: 2}
    return mapping[pred]
```